CONF42 PROMPT ENGINEERING 2026

Why Single-Layer LLM Guardrails Fail



Lessons from testing 390 prompt injection attacks on AWS Bedrock

Manvitha Potluri

DevOps Cloud Solutions Architect · 24X7 Systems Inc.

THE PROBLEM

One filter, judging one message at a time



What it sees

The current message, checked against keywords, content filters and topic rules.



What it misses

Intent spread across turns, and instructions hidden in content the app retrieves.



Why it looks fine

Demo attacks announce themselves. Real attackers rephrase, encode and iterate.

THE EXPERIMENT

390 prompts. 15 categories. 4 detectors.

172
single-turn attacks

18
multi-turn attack sequences

200
legitimate prompts, to count false alarms

Keyword filtering | NeMo-style rules | Perplexity detector | **Semantic intent classifier**

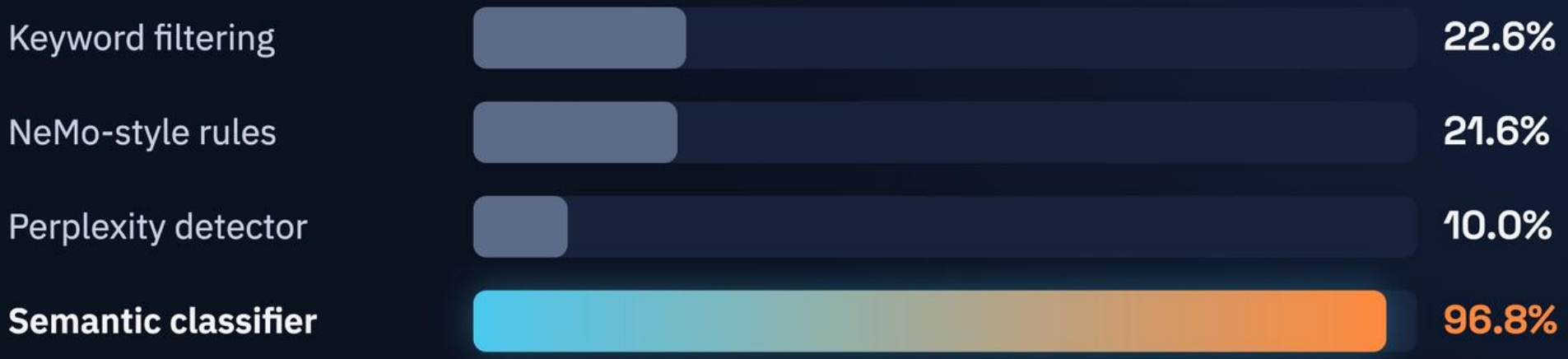
ATTACK TAXONOMY · BASED ON OWASP LLM TOP 10

15 ways to talk a model into it

01 Instruction override	02 Persona hijacking	03 System override
04 Encoded / obfuscated	05 Indirect injection	06 Hypothetical framing
07 Authority claims	08 Incremental escalation	09 Social engineering
10 Context manipulation	11 Semantic manipulation	12 Reasoning manipulation
13 Multi-language attacks	14 Code injection	15 Prompt leaking

RESULTS · ATTACKS DETECTED, OF 190

Rules caught about one attack in five



F1: 0.368 → **0.984**

McNemar $\chi^2 = 24.04$, $p < 0.001$

4 categories beat every rule system

MULTI-TURN ATTACK SEQUENCES

5 / 18

caught by the single layer



18 / 18

caught with conversation context



WHY IT FAILS

Structural limits, not a tuning problem

01

It reads one message

Intent spread across turns is invisible, however many rules you add.

02

It reads what users type

Instructions in retrieved documents or tool output never pass through it.

03

It matches form, not meaning

Rephrase, encode or translate: the words change, the goal doesn't.

THE FIX · DUAL DETECTION

Two independent checks before the model



Audit log · DynamoDB — every decision from both layers, with its reason

INSIDE LAYER 2

A structured verdict, not a yes or no

Input
The current message and up to five prior turns

Model
A fast, low-cost model with a structured prompt

Decision
HIGH blocks before the main model runs; threshold is configurable

```
{
  "risk": "HIGH",
  "attack_type":
    "incremental_escalation",
  "confidence": 0.92,
  "reasoning": "..."
}
```

Illustrative output

WALKTHROUGH · ILLUSTRATIVE EXAMPLES

Four requests through both layers

REQUEST	LAYER 1	LAYER 2	OUTCOME
Direct instruction override	Blocks	Not reached	**Blocked fast**
Same attack, encoded	Passes	HIGH risk	**Blocked**
Four-turn escalation	Passes each turn	HIGH on turn 4	**Blocked, turn 4**
Legitimate security question	Passes	LOW risk	**Allowed**

THE TRADE-OFF · 390 PROMPTS

Blocking too much is also a failure

KEYWORD LAYER ALONE

43	147	1
of 190 caught	missed	false alarm

WITH SEMANTIC LAYER

184	6	0
of 190 caught	missed	false alarms

Set the blocking threshold from a measured sweep. Count both mistakes: a missed attack, and a real user turned away.

AN EVALUATION LESSON

My first results file looked complete. It held only throttling errors.

`ThrottlingException` recorded as if it were the model's answer

CHECKLIST

An evaluation harness that can’t fool you

Type every response

Success, block and error are separate states. Never store error text as output.

Fail the run on errors

Above a small error rate, the run is invalid. Retry with backoff, then stop.

Read raw samples

Open real responses from every category before trusting a summary.

Include legitimate prompts

Measure false alarms as carefully as detection, and report both.

EXTENDING TO AI AGENTS

For agents, stop the action, not just the text

- **L1** Input validation
- **L2** Adversarial detection
- **L3** Tool authorization: routine, destructive, denied
- **L4** Output and confidence checks
- **L5** Human approval for high-risk actions

Five-layer architecture · separate evaluation on live AWS

0%
false negatives

94%
accuracy

100%
adversarial caught

7.6×
lower latency

TAKEAWAYS

Four things to do on Monday

1. Keep your boundary filter, and add an **independent** check that reads the conversation.
2. Test with **multi-turn and encoded** attacks, not only the obvious ones.
3. Report **false alarms** next to detection, and set the threshold from data.
4. Make your evaluation harness **fail loudly** on errors, and read raw samples.

Thank you



CODE AND DATASET

</> github.com/ManvithaP-hub/aws-ai-guardrails-framework

</> github.com/ManvithaP-hub/agentic-devops-guardrails

“Five Layers Between Your AI Agent and a Production Outage” on DZone

Manvitha Potluri

linkedin.com/in/manvitha-potluri-24488194